

Adestramento de modelos de IA

Laura Cabaleiro Pintos & Pablo Chantada Saborido

Marcelo Ferreiro Sánchez & José Romero Conde

Curso 2025–2026

Índice

1. Introducción	2
2. Dataset empregado	2
3. GAN	2
3.1. Motivación e formulación	2
3.2. Arquitectura implementada	3
3.3. Resultados da clasificación binaria co discriminador	4
3.4. GAN como xerador de datos sintéticos	7
4. AdaBoost	7
4.1. Clasificación binaria	7
4.2. Clasificación multiclase	10
4.3. Dataset Aumentado	12
5. Comparativa	12
6. Conclusións	13
7. Traballo Futuro	14

1. Introducción

Neste traballo empréganse dous modelos de intelixencia artificial sobre o dataset CIC-IDS2017 co obxectivo de estudar a súa aplicación nun sistema de detección de intrusións a nivel de rede. Os modelos escollidos foron AdaBoost [1] como modelo base e GAN [2] como modelo complexo. O obxectivo principal da práctica é resolver un problema de clasificación binaria, no que cada fluxo debe etiquetarse como **BENIGN** ou **ATTACK**. Ademais, como extensión, tamén se analiza o comportamento de AdaBoost no escenario multiclase, empregando as etiquetas orixinais do dataset.

Nesta segunda parte da práctica non se realiza de novo o preprocesado completo, senón que se parte do conxunto de datos xa preparado na entrega anterior. O traballo céntrase, por tanto, no adestramento, avaliación e comparación dos modelos, así como na análise das dificultades atopadas e das prestacións acadadas en cada caso.

2. Dataset empregado

O dataset empregado neste traballo procede do preprocesado realizado na primeira práctica. Partiuse da versión completa **TrafficLabelling** de CIC-IDS2017, sobre a que se eliminaron filas baleiras, valores nulos, valores infinitos, columnas duplicadas ou sen información útil e variables con alto risco de introducir sesgo, como identificadores de fluxo, IPs e marcas temporais.

Tras a limpeza inicial, o conxunto útil quedou en 2 827 876 fluxos. As variables categóricas principais, como portos e protocolo, foron codificadas numericamente, agrupando portos pouco frecuentes nunha categoría común. As variables numéricas foron normalizadas e posteriormente realizouse unha selección de características apoiada en PCA, mantendo finalmente 52 características predictoras.

Debido ao forte desbalanceo entre clases, aplicouse unha combinación de *undersampling* nas clases maioritarias e SMOTE nas clases minoritarias. O dataset final usado nesta práctica quedou composto por 182 000 mostras. A partición realizouse de forma estratificada cun reparto 70/15/15, obtendo 127 400 mostras de adestramento, 27 300 de validación e 27 300 de test.

No escenario binario, a etiqueta **BENIGN** codificouse como clase negativa e todas as demais etiquetas como clase positiva **ATTACK**. Esta mesma partición foi empregada tanto para AdaBoost como para a GAN.

3. GAN

3.1. Motivación e formulación

A GAN empregouse como modelo complexo da práctica. Durante o desenvolvemento probáronse varias formulacións.

Nun primeiro intento, a GAN formulouse como un detector de anomalías. Para iso, o discriminador adestrábase só con exemplos **BENIGN** do conxunto de adestramento balanceado e con mostras sintéticas xeradas a partir de ruído. A idea era que o discriminador aprendese a recoñe-

cer tráfico benigno e que, na avaliación, os fluxos cun score baixo fosen considerados anomalías ou ataques.

Despois probouse unha segunda variante do mesmo enfoque, empregando un conxunto máis amplo de exemplos benignos extraídos do dataset orixinal. Esta segunda proba foi tamén exploratoria e serviu para comparar o comportamento da GAN cando se lle fornecían só mostras benignas do conxunto balanceado fronte ao caso no que se empregaban moitos máis exemplos benignos procedentes do conxunto inicial.

A versión final implementada foi distinta. O discriminador recibe mostras reais do conxunto de adestramento e optimízase coa súa etiqueta binaria: **BENIGN** como clase negativa e **ATTACK** como clase positiva. Ademais, tamén recibe mostras sintéticas xeradas pola GAN, que se empregan como exemplos artificiais coa etiqueta 0 durante o adestramento do discriminador. Pola súa banda, o xerador optimízase para producir mostras que o discriminador clasifique como 1.

Polo tanto, a versión final non é unha GAN non supervisada clásica nin unha GAN condicionada. É unha formulación adversaria híbrida: o discriminador aproveita as etiquetas binarias reais como obxectivo da perda, pero a etiqueta non se introduce como entrada da rede.

3.2. Arquitectura implementada

A arquitectura final é deliberadamente sinxela. O xerador recibe un vector latente de tamaño z_dim e produce unha mostra sintética co mesmo número de características ca o dataset. O discriminador recibe unha mostra do espazo de entrada e devolve un único logit para a clasificación binaria.

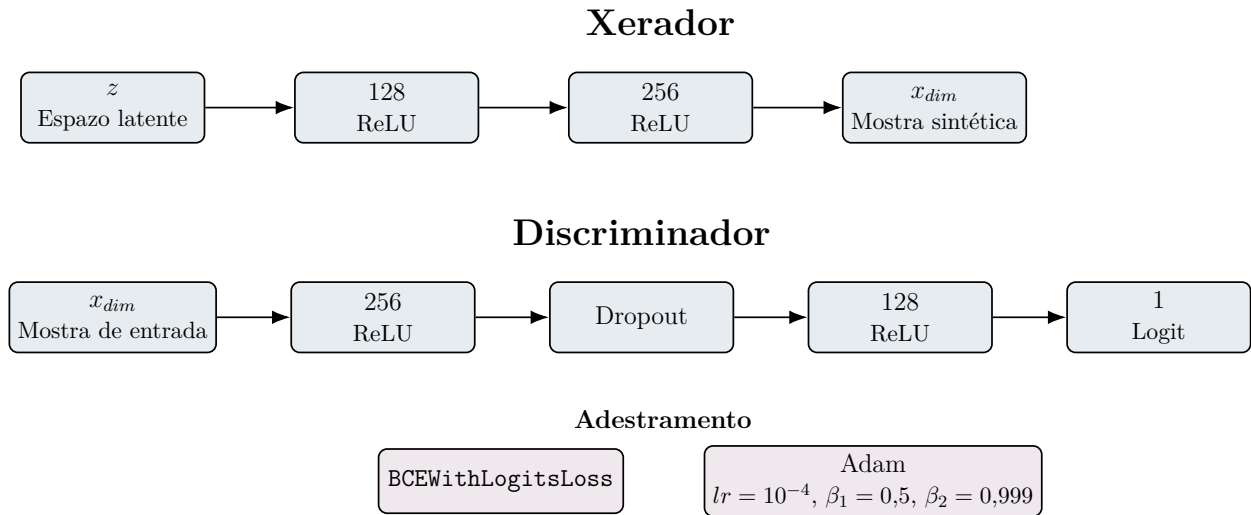


Figura 1: Arquitectura da GAN empregada. O xerador transforma un vector latente nunha mostra sintética co mesmo número de características ca entrada, mentres que o discriminador devolve un único logit para a clasificación binaria.

Tamén se parametrizaron o *dropout*, o tamaño do lote, a frecuencia de avaliación, o número de *workers* e a dimensión do espazo latente. Isto permitiu facer probas curtas para verificar o funcionamento da tubaxe e probas longas para intentar estabilizar o adestramento.

3.3. Resultados da clasificación binaria co discriminador

As primeiras execucións conservadas da GAN foron probas curtas de só 6 épocas, realizadas para comprobar que a tubaxe funcionaba correctamente. Nesas probas o modelo aínda non converxía: a AUC-ROC de validación quedaba arredor de 0,74–0,78 e a accuracy arredor de 0,45–0,47. Estes valores indican que o adestramento adversario necesitaba moitas máis épocas para estabilizarse.

Posteriormente realizouse un adestramento longo, no que se observaron métricas de validación moito máis altas arredor das épocas 2490–2530. Nese intervalo acadáronse valores de AUC-ROC arredor de 0,94 e unha accuracy próxima a 0,90. Isto mostra que a arquitectura era capaz de aprender unha fronteira útil, pero cun custo de adestramento moi superior ao de AdaBoost.

Nas probas curtas empregouse `z_dim=6`, `dropout=0.01`, `batch_size=512` e `lr=0.0001`. A diferenza principal entre execucións foi o dispositivo empregado e o número de *workers*.

Táboa 1: Resumo dos resultados dispoñíbeis da GAN en validación

Execución	Épocas	Mellor AUC-ROC	Mellor accuracy	Lectura
Proba curta MPS	6	0,7700	0,4683	Proba funcional, pero sen converxencia.
Proba curta CUDA, <code>workers=8</code>	6	0,7811	0,4653	Mellora lixeira en AUC, aínda con accuracy baixa.
Proba curta CUDA, <code>workers=2</code>	6	0,7832	0,4519	Resultado semellante ás demais probas curtas.
Adestramento longo	~2500	0,9356	0,9017	Converxencia clara tras un adestramento prolongado.

Cómpre sinalar que estas métricas corresponden a validación. Non se conserva unha avaliación final da GAN sobre test co mesmo nivel de detalle ca AdaBoost, polo que a comparación cuantitativa entre ambos modelos debe facerse con cautela.

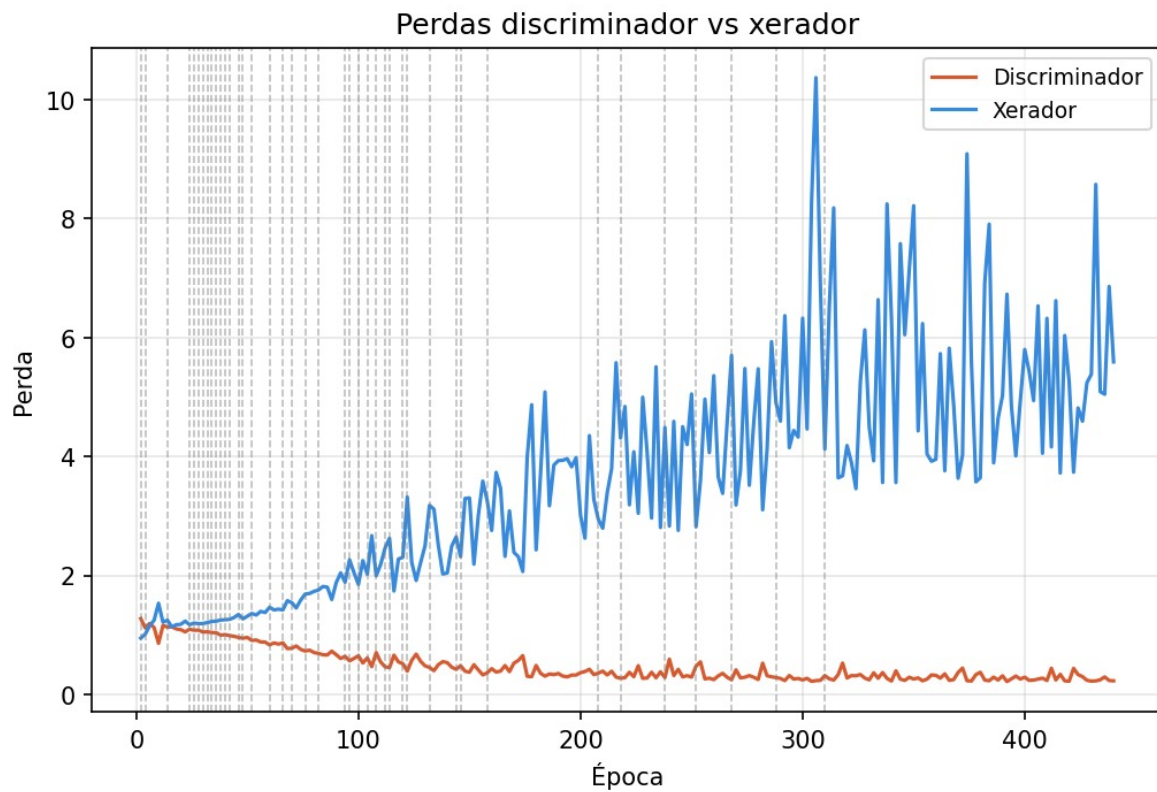


Figura 2: Evolución das perdas do discriminador e do xerador durante o adestramento longo da GAN. Obsérvase que a perda do discriminador diminúe progresivamente mentres que a do xerador tende a aumentar e presenta oscilacións considerábeis, algo habitual no adestramento adversario. As liñas verticais marcan épocas nas que se rexistraron melloras do mellor valor de validación.

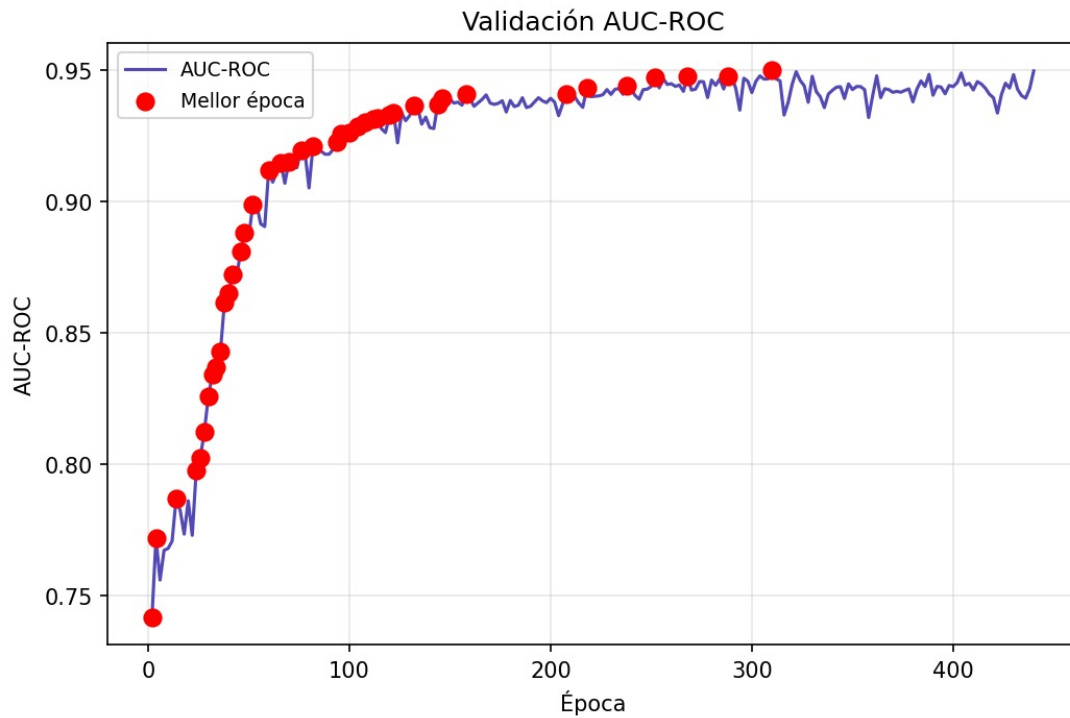


Figura 3: Evolución da AUC-ROC de validación ao longo do adestramento da GAN. Os puntos vermellos marcan as épocas nas que se mellorou o mellor valor acadado ata ese momento. Apréciase unha mellora rápida nas primeiras épocas e unha estabilización posterior arredor de 0,94–0,95.

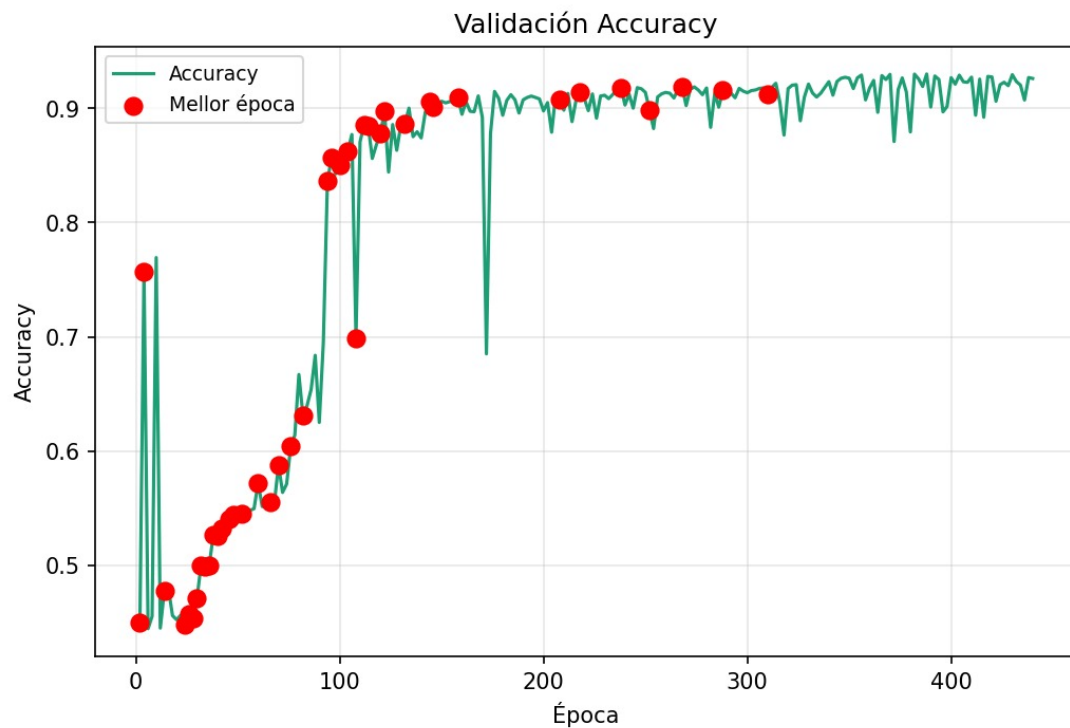


Figura 4: Evolución da *accuracy* de validación ao longo do adestramento da GAN. Os puntos vermellos indican as épocas nas que se acadou unha mellora do mellor valor ata o momento. Tras unha fase inicial inestábel, a *accuracy* estabilízase arredor de 0,90.

As gráficas anteriores amosan o comportamento típico dun adestramento adversario. Nas pri-

meiras épocas obsérvase unha mellora rápida da AUC-ROC e da *accuracy* de validación, mentres que posteriormente as curvas se estabilizan arredor de valores altos. Ao mesmo tempo, as perdas do xerador e do discriminador presentan oscilacións, o que reflicte a dificultade de equilibrar ambas redes durante o adestramento.

3.4. GAN como xerador de datos sintéticos

Un dos engadidos máis interesantes do traballo foi a implementación dunha tubaxe de *data augmentation*. O script `xerar_dataset.py` carga o mellor xerador dispoñíbel, xera novas mostras en lotes e reconstrúe un novo `train.parquet`, mantendo `val.parquet` e `test.parquet` sen cambios.

O parámetro `p` controla a proporción de mostras reais que se quere manter no novo conxunto de adestramento. Nunha das execucións conservadas xeráronse 127 400 mostras sintéticas adicionais e o novo `train` quedou en 254 800 filas.

Hai que sinalar que o script etiqueta explicitamente as mostras xeradas como `SINTETICO` no `parquet` resultante. Polo tanto, o efecto exacto desta augmentación depende de como se recargue ese dataset nas fases posteriores de adestramento e de se se traballa en binario ou en multiclase. A tubaxe quedou implementada e funcional, pero non se chegou a pechar unha campaña experimental homoxénea que permitise cuantificar con precisión canto mellora AdaBoost cando se adestra con este dataset aumentado.

4. AdaBoost

AdaBoost foi o modelo base escollido para a práctica. Trátase dun modelo supervisado sinxelo, rápido de adestrar e axeitado para datos tabulares, polo que serve como referencia para comparar fronte á GAN.

Realizouse unha primeira procura mediante `GridSearchCV`, tanto no caso binario como no multiclase. Esta proba permitiu comprobar que AdaBoost funcionaba correctamente sobre o dataset preprocesado e obter unha primeira estimación do rango de hiperparámetros útil.

Na versión final empregouse optimización bayesiana para automatizar a procura de hiperparámetros. O espazo de busca incluíu `n_estimators` entre 15 e 500 e `learning_rate` entre 0.0001 e 2. A execución final realizouse con 15 puntos aleatorios iniciais e 40 iteracións adicionais de optimización. A función obxectivo foi o F1 micro en validación.

O script permite executar AdaBoost tanto en binario como en multiclase. Se se activa o flag `-binary`, empréganse as etiquetas binarias; se non, úsanse as etiquetas multiclase orixinais. Isto permitiu avaliar o mesmo modelo nos dous escenarios sen modificar o código.

4.1. Clasificación binaria

No escenario binario, AdaBoost obtivo o mellor resultado documentado da práctica en termos de relación custo-rendemento. A mellor configuración atopada no notebook foi `n_estimators = 300` e `learning_rate = 1.5`. Con esa configuración acadouse un F1 de validación cruzada de 0,9251 e, sobre o conxunto de test, unha *accuracy* de 0,8872 e un F1 de 0,9265.

O reparto do problema binario no dataset final foi o seguinte: 35 000 mostras **BENIGN** e 92 400 **ATTACK** no adestramento, e 7 500 / 19 800 tanto en validación como en test. O modelo amosou un *recall* moi alto para **ATTACK** (0,98), sacrificando parte do *recall* da clase **BENIGN** (0,64). Nun contexto de ciberseguridade, esta compensación é aceptábel, xa que adoita ser preferíbel asumir máis falsos positivos antes que deixar pasar tráfico malicioso.

Táboa 2: Resultados binarios de AdaBoost no dataset orixinal

Configuración	CV F1	Accuracy test	F1 test	Nota
300 est., lr=1.5	0,9251	0,8872	0,9265	<i>Recall</i> ATTACK = 0,98

As figuras seguintes recollen a evolución da busca bayesiana no caso binario. Obsérvase a converxencia do máximo acumulado e a localización das mellores combinacións no espazo de parámetros.

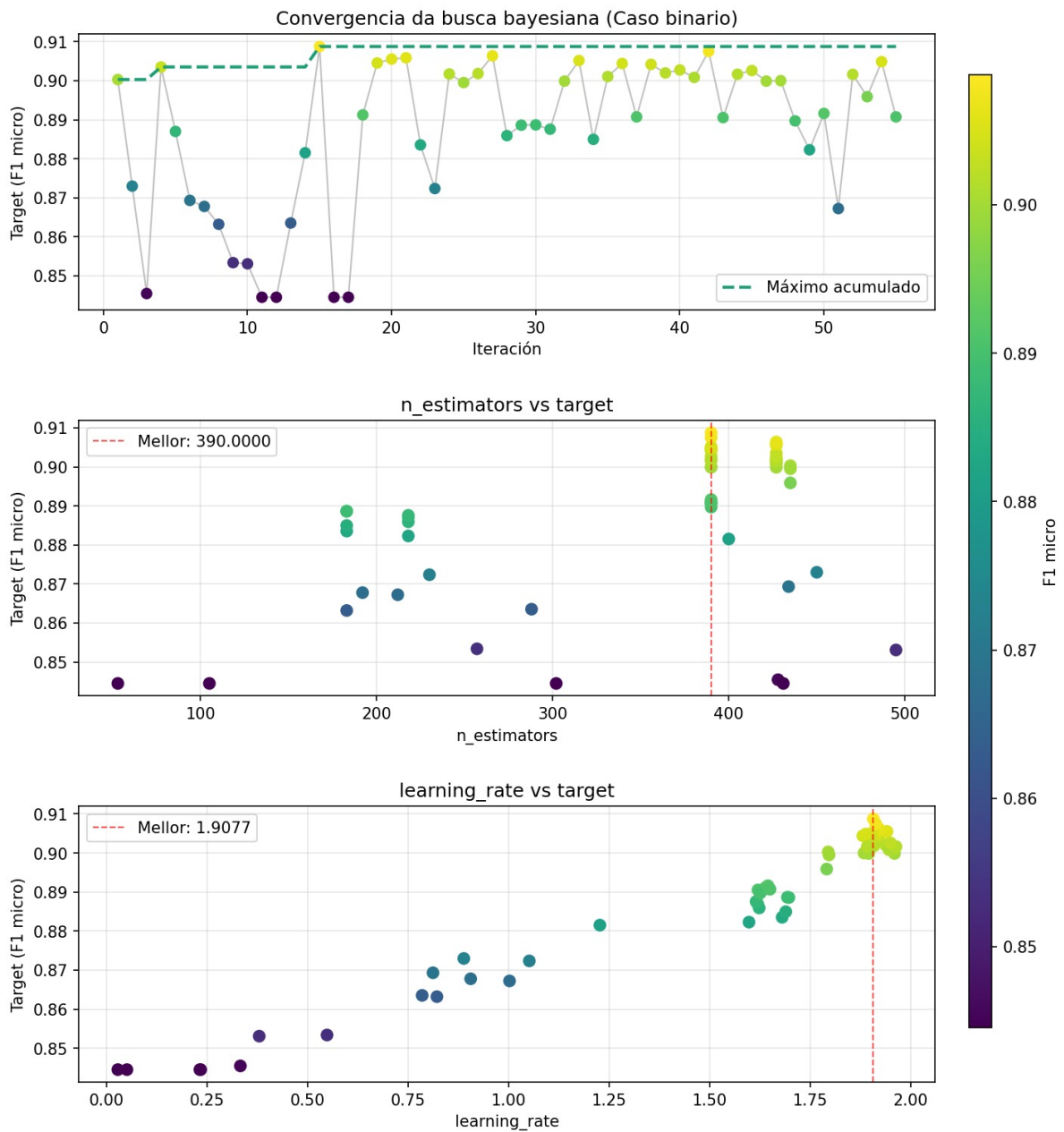


Figura 5: Convergencia da busca bayesiana de AdaBoost no caso binario. O mellor valor aproxímase a F1 micro $\approx 0,909$, cun `n_estimators` próximo a 390 e `learning_rate` arredor de 1,908.

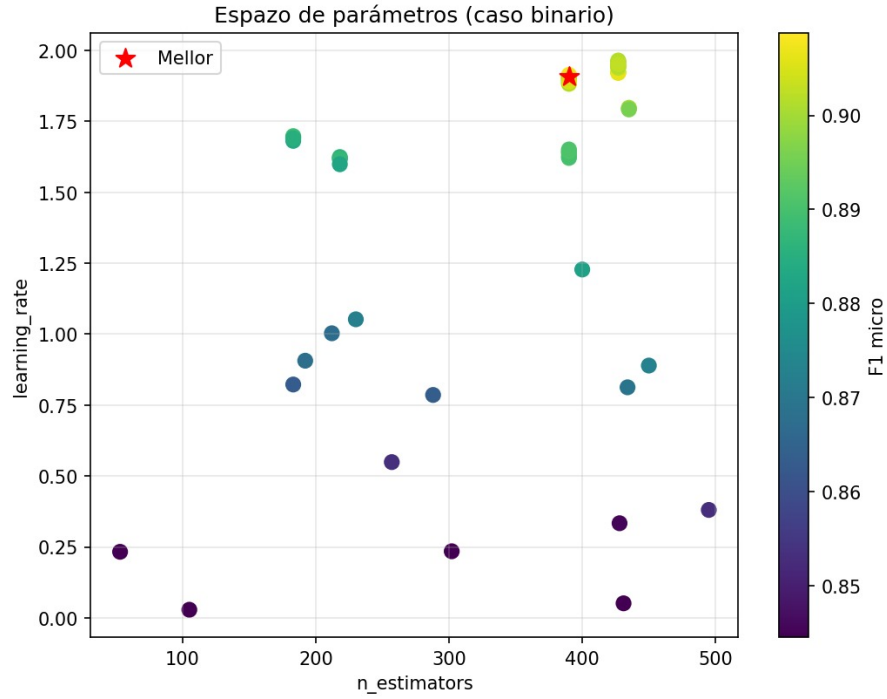


Figura 6: Espazo de parámetros explorado para AdaBoost no caso binario. A concentración de puntos de maior calidade aparece na zona de `learning_rate` alto e número de estimadores medio-alto.

4.2. Clasificación multiclase

Tamén se estudou unha extensión multiclase. Neste escenario mantéñense as etiquetas orixinais do dataset, polo que o problema é máis difícil ca no caso binario: ademais de distinguir tráfico benigno fronte a ataque, o modelo debe identificar o tipo concreto de tráfico ou ataque.

O mellor modelo multiclase obtivo unha *accuracy* de 0,4966 e un *weighted F1* arredor de 0,44. As clases maioritarias e algunhas firmas específicas como **SSH-Patator** ou **FTP-Patator** tiveron un comportamento aceptábel, pero as clases moi raras seguiron moi penalizadas mesmo despois do rebalanceo.

Nunha execución curta reproducíbel vía script obtívose *accuracy* = 0,4286, *macro F1* = 0,3266 e *weighted F1* = 0,4379. Esta proba non se toma como mellor resultado final, senón como unha evidencia adicional da dificultade do escenario multiclase e da importancia do orzamento dedicado á procura de hiperparámetros.

As figuras seguintes recollen a evolución da busca bayesiana no caso multiclase.

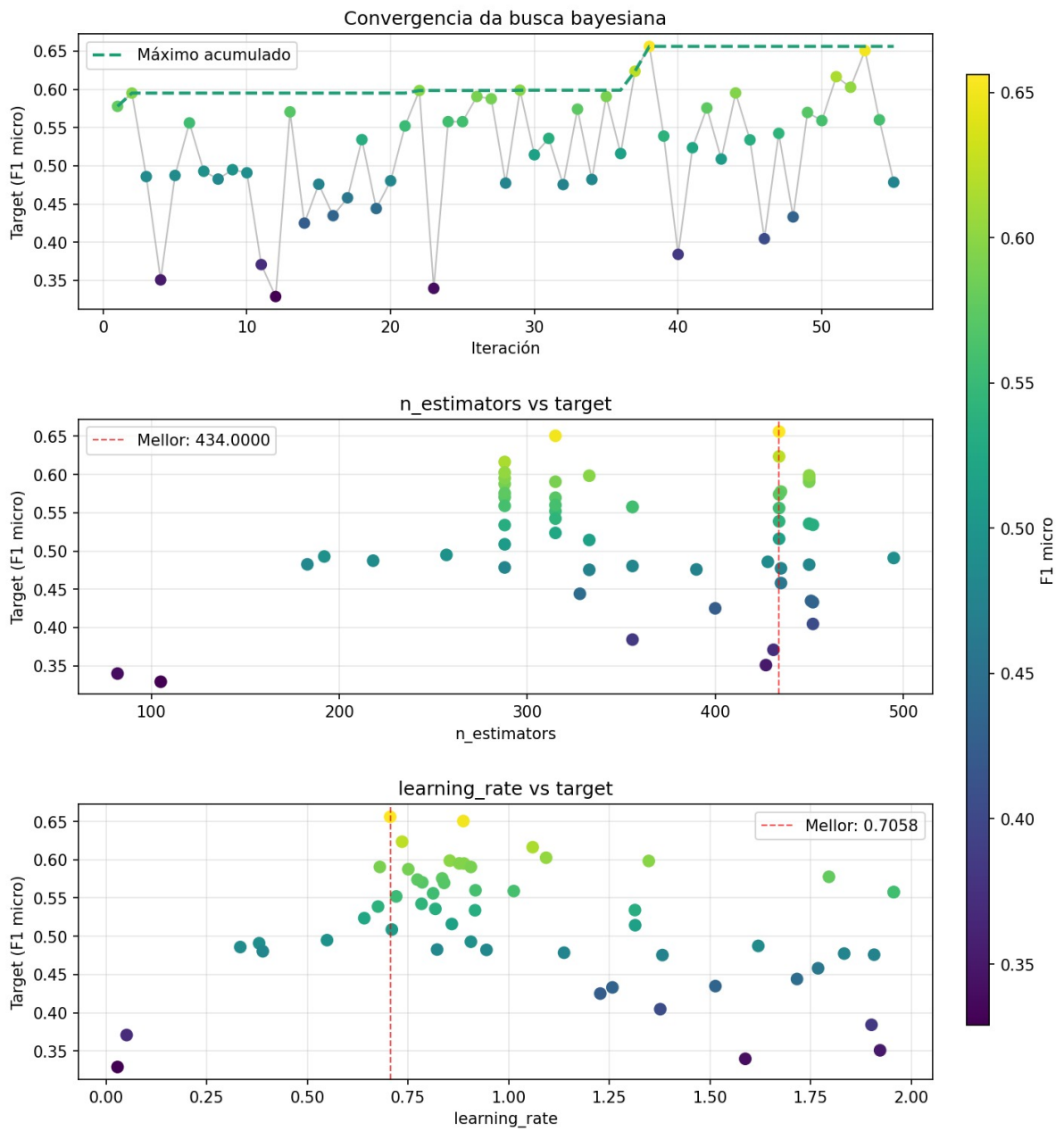


Figura 7: Convergencia da busca bayesiana de AdaBoost no caso multiclase. Obsérvase un mellor valor arredor de F1 micro $\approx 0,656$, con `n_estimators` próximo a 434 e `learning_rate` arredor de 0,706.

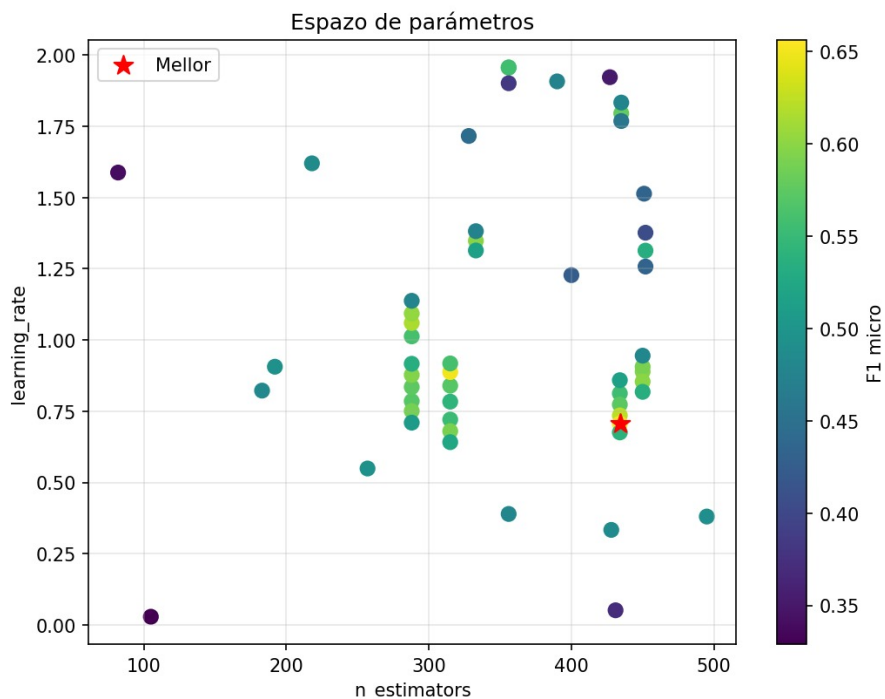


Figura 8: Espazo de parámetros explorado para AdaBoost no caso multiclase. A estrela sinala a mellor combinación atopada pola optimización bayesiana.

4.3. Dataset Aumentado

A continuación de empregar a GAN como xerador, deseñouse un fluxo para adestrar AdaBoost sobre un dataset ampliado artificialmente. O obxectivo consistía en manter unha parte das mostras reais e engadir un número controlado de mostras sintéticas xeradas pola GAN, de maneira que o conxunto de adestramento fose máis grande e potencialmente máis rico.

Nunha das execucións conservadas obtívose un novo dataset aumentado con 254 800 filas en adestramento, fronte ás 127 400 do conxunto orixinal. Posteriormente, este novo conxunto podía empregarse para volver adestrar AdaBoost sobre os datos xerados.

Esta extensión resulta interesante desde o punto de vista experimental porque permite converter a GAN nunha técnica auxiliar e non só nun clasificador. Porén, a evidencia conservada non é suficiente para ofrecer unha comparación final pechada entre AdaBoost adestrado co dataset orixinal e AdaBoost adestrado co dataset aumentado. O que si pode afirmarse é que a infraestrutura para facelo quedou completamente implementada.

5. Comparativa

A comparación entre ambos modelos debe facerse con certa cautela, porque a trazabilidade experimental non é totalmente simétrica. De AdaBoost consérvanse métricas finais de test moi claras; da GAN consérvanse sobre todo trazas de validación e saídas de consola, ademais do fluxo de xeración sintética. Aínda así, o material dispoñíbel permite establecer varias conclusións.

En primeiro lugar, AdaBoost foi o mellor modelo base do traballo. A súa principal vantaxe é a eficiencia: adéstrase rápido, é doado de optimizar e ofrece un rendemento moi competitivo

no problema binario. Ademais, a interpretación das súas métricas é directa e permite construír unha memoria máis limpa e reproducíbel.

En segundo lugar, a GAN xustifica a súa presenza como modelo complexo precisamente porque introduce dificultades reais de adestramento. As probas curtas demostraron que a converxencia non chega en poucas épocas; o *dropout*, o hardware e o número de iteracións inflúen moito máis do esperado. Por outra banda, cando se lle dedica tempo dabondo, o discriminador consegue métricas altas en validación e o xerador abre a porta á creación de novos datasets.

Táboa 3: Comparativa cualitativa entre AdaBoost e GAN

Aspecto	AdaBoost	GAN	Lectura
Custo de adestramento	Baixo e rápido.	Moi alto; require milleiros de épocas.	AdaBoost é claramente máis eficiente.
Rendemento documentado	Moi bo en binario; discreto en multiclase.	Malo ao inicio, competitivo tras converxer.	A GAN ten máis teito, pero tamén máis risco.
Estabilidade	Alta.	Moi sensíbel a hiperparámetros e hardware.	A GAN require máis axuste fino.
Interpretabilidade	Maior facilidade para xustificar o comportamento global.	Menor interpretabilidade.	AdaBoost é mellor <i>baseline</i> académico.
Valor engadido	Non xera datos novos.	Pode xerar mostras sintéticas e ampliar o train .	A GAN achega utilidade adicional.

En resumo, se só se valora a relación entre simplicidade, velocidade e rendemento final, AdaBoost é a mellor opción. Se se valora tamén a experimentación, a capacidade de xerar datos e o interese técnico do proceso, a GAN aporta un valor que vai máis alá da simple clasificación.

6. Conclusións

O traballo permitiu completar o obxectivo principal da práctica: adestrar e comparar dous modelos diferentes sobre CIC-IDS2017, partindo do preprocesado realizado na primeira entrega e analizando os resultados obtidos.

AdaBoost consolidouse como unha solución base moi forte no escenario binario. Coa configuración axeitada ofreceu métricas altas, un tempo de execución reducido e unha interpretación clara dos resultados. Por iso pode considerarse o modelo máis robusto do traballo.

A GAN foi máis esixente. As probas iniciais non chegaron a converxer, pero os adestramentos longos demostraron que o discriminador podía acadar resultados competitivos en validación. Ademais, o feito de reutilizar o xerador para producir novas mostras converte a GAN nunha ferramenta con utilidade máis ampla dentro do pipeline experimental.

A conclusión global é que o modelo sinxelo ofrece a mellor relación custo-rendemento, mentres

que o modelo complexo introduce flexibilidade e posibilidades experimentais adicionais. Esa combinación fai que a comparativa entre ambos sexa rica e académicamente xustificábel.

7. Traballo Futuro

A partir dos resultados obtidos, as principais liñas de mellora serían as seguintes:

- Avaliar a GAN sobre o conxunto de test co mesmo nivel de detalle ca AdaBoost, incluíndo *accuracy*, *precision*, *recall*, F1-score, AUC-ROC e matriz de confusión.
- Realizar unha campaña experimental homoxénea para medir o efecto do dataset aumentado pola GAN sobre AdaBoost, comparando sempre contra o mesmo conxunto de validación e test.
- Ampliar a avaliación multiclase de AdaBoost con máis execucións e métricas macro, para analizar mellor o comportamento nas clases menos representadas.
- Comparar a formulación adversaria híbrida implementada coa versión baseada só en tráfico benigno, co obxectivo de comprobar cal das dúas resulta máis axeitada para detección de anomalías.
- Explorar variantes máis específicas da GAN, como GANs condicionais ou unha GAN independente por clase, no caso de querer abordar o problema multiclase con maior profundidade.

Referencias

- [1] Y. Freund and R. E. Schapire, “Experiments with a new boosting algorithm,” pp. 148–156, 1996.
- [2] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative adversarial nets,” in *Advances in Neural Information Processing Systems*, vol. 27, 2014.